

COMP90038 Algorithms and Complexity

Algorithms and Data Structures

Junhao Gan

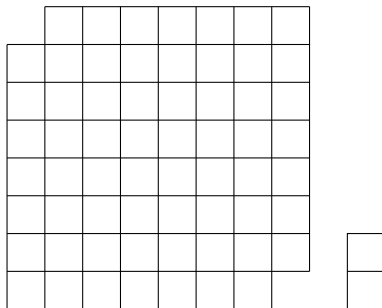
Week 1 Lecture 2

Semester 1, 2026

Approaching a Problem

Can we cover this board with 31 tiles of the form shown?

This is known as the **Mutilated Checkerboard Problem**.

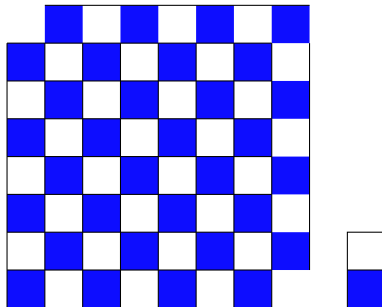


There are only finitely many ways we can arrange the 31 tiles, so there is a brute-force (yet very inefficient) way of solving the problem.

Question: Can you come out with a more efficient solution?

Transform and Conquer? Use Abstraction?

Can we cover this board with 31 tiles of the form shown?



Hint: Using the way the squares are coloured helps.

Why can we quickly determine that the answer is **NO**?

Algorithms and Data Structures

Algorithms: for solving problems and transforming data.

Data structures: for storing data; arranging data in a way that suits an algorithm.

- Linear data structures: [Arrays](#), [Stacks](#) and [Queues](#)
- Trees and Graphs
- Dictionaries

Which data structures are you familiar with?

Primitive Data Structures: Array

An array corresponds to a sequence of *consecutive* cells in memory.

Depending on programming language: $A[0]$ up to $A[n-1]$, or $A[1]$ up to $A[n]$.

Locating a cell, and storing or retrieving data at that cell is very fast.

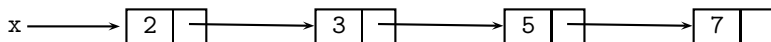
The downsides of an array are that:

- the length n has to be *known in advance* and is *fixed*;
- maintaining a *contiguous* bank of cells with information can be difficult and time-consuming.

6	9	2	3	7	5	8
0	1	2	3	4	5	6

42148	6
42149	9
42150	2
42151	3
42152	7
42153	5
42154	8

Primitive Data Structures: Linked List



A collection of objects with links to one another, possibly in different parts of the computer's memory.

Each “*node*” has two attributes: a “*val*” which is its value, and a “*next*” which points to the rest of the list.

Here *x* corresponds to address 42154.

42148	3
42149	42150
42150	5
42151	42156
42152	
42153	
42154	2
42155	42148
42156	7
42157	0

Often we use a *dummy head node* that points to the first object, or to a special *null* object that represents an empty list. This makes it easier to write functions that insert or delete elements.

Inserting and deleting elements is very fast: just move a few links around.

However, finding the i^{th} element can be time-consuming.

Iterative Processing

Walk through the array (of length n), or through the linked list.

For example, to locate item x .

```
function find(A,x,n)
  j := 0
  while j < n
    if A[j] == x
      return j
    j := j+1
  return -1
```

```
function find(head,x)
  p := head
  while p != null
    if p.val == x
      return p
    p := p.next
  return null
```


Recursive Processing

Solve the problem for a *sub-instance* and use the solution to solve the *full instance*.

```
function find(A,x,lo,hi)
  if lo > hi
    return null
  else if A[lo] == x
    return lo
  else
    return find(A,x,lo+1,hi)
```

Initial call: find(A,x,0,n-1)

```
function find(p,x):
  if p == null
    return p
  else if p.val == x
    return p
  else
    return find(p.next,x)
```

Initial call: find(head,x)

We will discuss **recursion** in detail in Week 3.

Abstract Data Types (ADT)

A collection of *data items*, and a family of *operations* that operate on that data.

Think of an ADT as a set of promises, or contracts.

We must *implement* these promises, but it is an advantage to separate the implementation of the ADT from the “concept”.

Good programming practice is to support this separation:

Nothing outside of the definitions of the ADT should refer to anything inside, except through function calls for the basic operations.

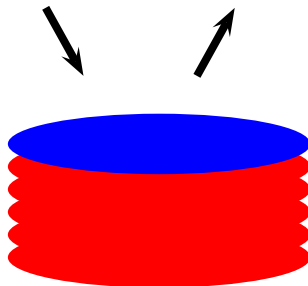
Such separation is also known as **Encapsulation**.

Fundamental Data Structures: Stack

Last-in-first-out (LIFO).

Operations:

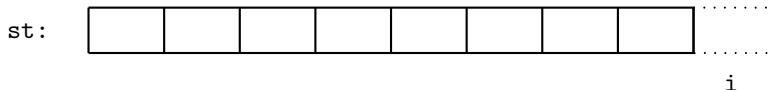
- CreateStack
- Push
- Pop
- Top
- EmptyStack?
- ...



Usually implemented as an ADT.

Stack Implementation with an Array

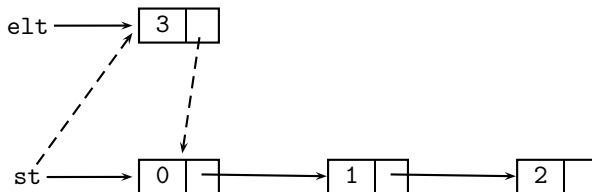
With an array:



Question: Is an array an appropriate data structure for implementing a stack? And why?

Stack Implementation with a Linked List

With a linked list (push):



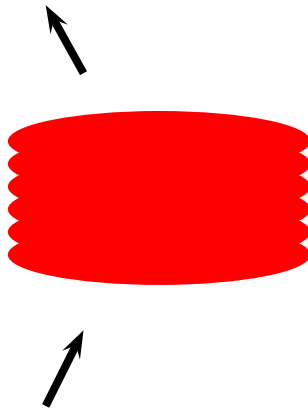
```
function PUSH( $x, st$ )  
   $elt \leftarrow \mathbf{new\ node}$   
   $elt.val \leftarrow x$   
   $elt.next \leftarrow st$   
   $st \leftarrow elt$   
  return  $st$ 
```

Fundamental Data Structures: Queue

First-in-first-out (FIFO).

Operations:

- CreateQueue
- Enqueue
- Dequeue
- Head
- EmptyQueue?
- ...



Other Data Structures

We shall meet many other (abstract) data structures, such as

- The priority queue
- Various types of “tree”
- Various types of “graph”

If you check out algorithm animation tools or advanced algorithm books, you will meet exotic data structures such as splay trees and skip lists.

Algorithm analysis—how to reason about an algorithm's resource consumption.